논문 리뷰

[강화학습] Playing Atari with Deep Reinforcement Learning DQN 논문 리뷰

letterhill | 2026. 3. 30. 23:43

Abstract

위 본문에서는 강화 학습을 사용하여 고차원 입력으로부터 직접 제어 정책을 성공적으로 학습하는 딥러닝 모델을 제시한다.
그 모델은 CNN 모델이며, 변형 Q-learning 으로 학습되었다
- input: raw pixels
- output: 미래에 받을 보상을 추정하는 value function가치함수
실험에서는 구조나 학습 알고리즘 조정 없이 Atari games에 해당 방법을 적용해보았다.
실험 결과, 이전의 모든 접근 방식보다 훌륭한 성능이며, 게임의 전문가를 능가할 정도의 성능을 보인다는 결과를 얻어냈다.

Introduction

오래 전부터, vision이나 speech같은 high-dimension sensory input으로부터 에이전트를 직접 제어하는 방법을 학습하는 것은 강화학습의 과제로 남아있었다. 대부분의 성공적인 강화학습 application들은 선형 가치 함수나 정책 표현과 결합된 수작업 특징에 의존해왔다. 분명히, 이러한 시스템의 성능은 특징 표현의 품질에 크게 의존한다.

수작업 특징$Hand-crafetedfeatures$란 어떤 것을 의미하는 걸까?

DQN 모델의 등장 전까지는 RL에서 에이전트가 게임 화면을 보고 바로 상황을 판단하는 것이 어려워서 연구자들이 정보들을 직접 가공컴퓨터가이해하기쉽도록해서 전달해주었다. 이 방식으로 추출한 feature를 hand-crafted features라고 한다.

구분	Hand-crafted Features	DQN
비유	밀키트 조리	원재료 조리
주체	사람이 직접 로직을 짜서 추출	에이전트가 스스로 학습하여 추출
의존성	사람이 뽑아준 특징의 품질에 크게 의존	데이터와 연산량에 의존

최근 딥러닝의 발전으로 raw 데이터로부터 고수준의 특징을 추출하는 것이 가능해져 이는 cv, 음성 인식 분야에 큰 발전을 불러왔다.
유사한 기술이 강화 학습RL에서도 효과적일지 자연스러운 궁금증이 떠오른다.
그러나, 강화학습은 딥러닝 관점에서 몇 가지 어려움딥러닝모델과의차이점이 있다.

구분	DL 알고리즘	RL 알고리즘
학습 데이터	대량의 수작업 라벨링 training data를 필요로 한다.	sparse, noisy, delayed 되는 보상 신호로부터 학습이 가능해야 한다. $*$행동 $-$ 보상사이의지연이매우클수있다.
데이터 간의 관계	데이터 샘플이 독립적	일반적으로 상호 연관성이 높은 상태들의 sequences
분포	고정된 분포를 가정	알고리즘이 새로운 학습 -> 데이터 분포가 변함

이 논문은 합성곱 신경망이 복잡한 RL 환경에서 raw 비디오 데이터로부터 성공적인 제어 정책 학습 보여준다.
이 네트워크는 Q-learning, SGD로 학습 -> 가중치 업데이트
correlated data와 non-stationary distribution 문제를 완화하기 위해, experience replay mechanism 무작위샘플링 -> 과거의 많은 행동에 걸쳐 학습 분포를 smooth하게 만든다.

실험:

ALE에 구현된 Atari 2600에 구현
Atari 2600: 도전 RL 고차원 visual input$210x160RGBvideoat50Hz$과 인간 player에게 어렵게 설계된 다양하고 흥미로운 과제 제공

Goal: 많은 게임을 성공적으로 학습하는 단일 신경망 에이전트 만들기

- 오직 video input, reward&termial signals, 그리고 가능한 action set만 제공

$humanplayer$와동일

- hand-designed feature가공정보 제공X

Background

MDP$MarkovDecisionProcess$

- 에이전트가 환경 $\mathcal{E}$와 상호작용하며, 행동(a_t), 관찰(x_t), 보상(r_t)을 주고 받는다.

- Partial Observability : 에이전트는 현재의 이미지만을 관찰하고 이는 부분적인 관찰이기 때문에 많은 emulator state는 인지적으로 혼동될 수 있다.

-> 즉, 현재의 화면 x_t로부터 현재의 상태를 완벽하게 이해하는 것은 불가능하다!

- s_t 재정의: 이를 해결하기 위해 현재 화면+시퀀스 $s_t = x_1, a_1, x_2, ..., a_{t-1}, x_t$ 를 하나의 '상태'로 재정의한다.

-> 시퀀스 s_t를 사용하여 표준 RL 방법을 MDP에 적용할 수 있다.

  * 시퀀스: 현재+과거의 행동 및 관찰화면

$Q*$

- 에이전트의 목표: 미래에 받을 보상을 최대화하는 방향으로 행동을 선택하는 것

우리는 미래에 받을 보상은 매 time-step마다 γ만큼의 할인율이 적용된다고 가정한다.

미래의 할인된 return값: $Rt = \sum_t^T \gamma^{t-t} r_t$

* T: 게임 종료 time-step

- 최적 행동 가치 함수 $Q*(s,a)$

  :어떤 전략을 따라서 시퀀스 s를 관찰하고 행동 a를 취하면 달성할 수 있는 최대 기대 보상

- $Bellmanequation$

최적 행동가치 함수는 $Bellmanequation$에 의해 다루어짐

Intuition

만약 가능한 모든 행동 a'의 수행 시점의 다음 time-step인 시퀀스 s'의 최적 가치 Q*

최적 전략: 가치 글 최대화 하는 행동 a'를 선택

RL 알고리즘의 기본 idea: Iterative update

- RL 알고리즘의 기본 idea: Bellman equation을 반복적 업데이틀 하여 행동 가치 함수를 추정

- 반복 알고리즘은 최적 행동 가치 함수로 수렴

Q-network

- 위의 방법은 행동 가치 함수가 각 시퀀스마다 별도로 추정되어 비효율적이라는 문제가 있다.
=> sol: function approximator기법을 사용하여 손실함수를 최소화
=> Q-network: theta 가중치를 가진 신경망 function approximaor

Related Work

1. TD-gammon

- RL과 self-play로 학습하여 인간 이상의 수준을 달성한 backgammon게임명 프로그램
- Q-learning과 유사한 model-free 강화 학습 알고리즘을 사용- 한 개의 은닉층으로 된 다중 선형 퍼셉트론을 활용한 가치 함수를 근사-> TD-gamman을 모방한 초기 시도들은 성공적X ; 해당 접근법은 backgammon 게임에서만 작동하는 특수한 경우

** backgammon: 주사위로 움직이는 레이싱 게임
+ Q-learning과 같은 model-free RL 알고리즘+nonlinear function approximators => Q-network의 발산

=> 해당 연구의 실패로, RL에서의 연구는 수렴이 보장된 linear function approximator을 사용했다.

2. NFQ$NeuralFittedQ-learning$

- RPROP 알고리즘을 사용하여 Q-net 업데이트, 배치 업데이트 사용
* DQN과의 차이점
- NFQ: Deep auto-encoder로 저차원 표현을 먼저 학습한 뒤, NFQ 적용 => 직접 입력 데이터를 학습하는 것이 아님!
- DQN: end-to-end 학습

Deep Reinforcement Learning

그렇다면, 본 논문의 핵심 알고리즘에 대해 자세히 알아보자.

Experience Replay

- Deep Q-learning 알고리즘에서 inner loop의 핵심 아이디어
- time-step마다 에이전트의 experience를 $replaymemory$에 저장하고 무작위로$randombatch$ 꺼내서 학습한다.
- experience $e_t = (s_t, a_t, r_t, S_{t+1})$

Deep Q-learning 알고리즘을 자세히 살펴보자.

Algorithm 1 Deep Q-learning with Experience Replay

Initialize replay memory $\mathcal{D}$ to capacity N
Initialize action-value function Q with random weights
for episode $= 1, M$ **do**
  Initialise sequence $s_1 = \{x_1\}$ and preprocessed sequenced $\phi_1 = \phi(s_1)$
  for $t = 1, T$ **do**
    With probability ϵ select a random action a_t
    otherwise select $a_t = \max_a Q^*(\phi(s_t), a; \theta)$
    Execute action a_t in emulator and observe reward r_t and image x_{t+1}
    Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$
    Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in $\mathcal{D}$
    Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from $\mathcal{D}$
    Set $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$
    Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ according to equation 3
  end for
end for

Initialization

1. experience를 저장하는 메모리 D 공간을 확보한다.
2. 신경망 Q를 초기화한다.

Iteration

1, ϵ-greedy policy: 최적의 행동만이 아닌, 가끔은 무작위로 행동을 선택할 수 있게 하는 방법
2. experience 저장
3. random sampling: 저장된 experiences 중 무작위로 샘플링한다.
4. y_j계산 : 현재시점t에서의행동으로얻은보상r_j$과 다음 상태에서 얻을 것으로 예상되는 보상의 최대치 $maxQ$ 를 더한다.

5. Gradient Descent: 예측값과 목표치 차이를 최소화하도록 신경망을 업데이트한다.

기존 Online Q-learning vs. Deep Q-learning

	Online Q-learning	Deep Q-learning
효율성 측면	현재시점, 다음시점의 데이터를 계속 저장하고 학습 => 비효율	한 번의 experience를 계속 꺼내서 재사용 가능 => 효율적
안정성 측면	연속된 데이터를 활용하여 서로 상관도가 너무 높음 => 학습이 불안정함	랜덤 샘플링 도입 => 샘플 간 상관관계 제거
policy	on-policy 학습 -> 현재의 파라미터 영향으로 편향된 학습	off-policy: experience replay -> 파라미터의 발산 방지

Proprocessing and Model Architecture

Atari game frame: 210x160 pixel color image

-> 그대로 학습 사용 시 엄청난 계산 비용 발생하기 때문에, input 차원을 축소하기 위한 전처리 과정이 필요하다.

Preprocessing

1) reduce # of channels: RGB -> gray

2) down sampling: 210x160 -> 110x84

3) img crop: 84x84

4) 함수 φ: 마지막 4 frames에 전처리를 적용하고 쌓아 Q-function의 input 생성

Parameterizing

기존 연구의 한계점:

기존 연구에서는 history-action pair를 신경망의 입력으로 사용하였으나 이는 각 행동의 Q-value 계산을 위해 개별적인 forward pass가 필요했기 때문에 행동의 수에 비례해 계산 비용이 linear하게 증가하는 단점이 있었다.

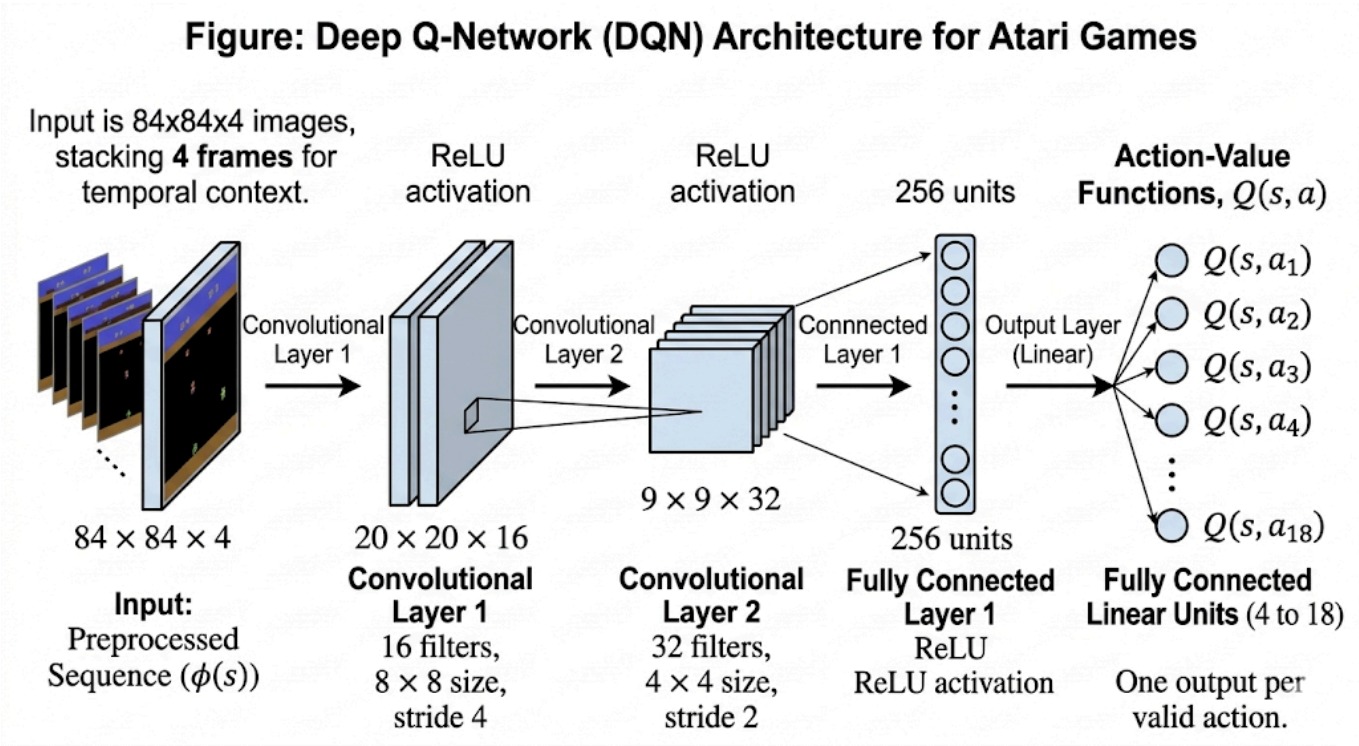
새로운 방법:

각 action에 대해 별도의 출력 unit이 있으며, 신경망에는 state 표현만 입력되는 구조를 도입

output: input state에 대한 개별 action의 예측 Q-value

=> 해당 구조의 장점: 네트워크를 통한 single forward pass만으로 주어진 상태에서 모든 가능한 행동의 Q-value가 계산 가능

Model Architecture



Experiments

- 7개의 게임으로 실험 진행
- 게임마다 점수의 magnitude가 다르므로 아래와 같은 reward를 설정하였다.
  * positive reward: +1
  * negative reward: -1

Hyperparameter

1 Optimizer: RMSProp

2 mini-Batchsize: 32

3 행동 정책: ϵ-greedy

- 초기 ϵ: 1 -> ϵ:0.1

4 총 학습 프레임: 10 million frames

5 총 사용 replay memory: one million frames

Frame-skipping technique

: 모든 프레임이 아닌 k번째 프레임에서만 관찰하고 행동을 선택하고, skip한 프레임에 대해서는 마지막 행동을 반복하는 기법

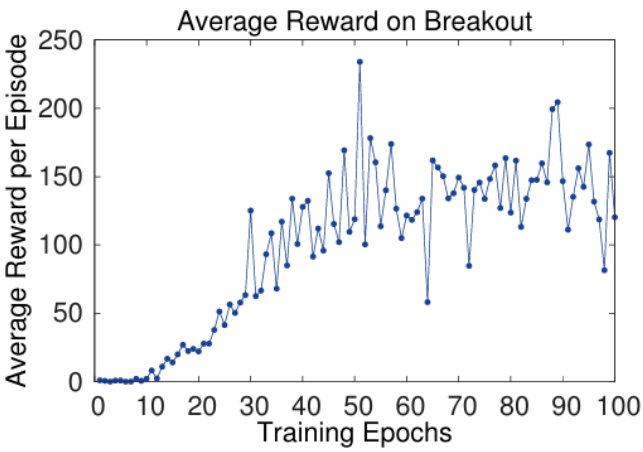
-> 실행 시간을 크게 늘리지 않고 k배 더 많은 게임 play 가능해짐

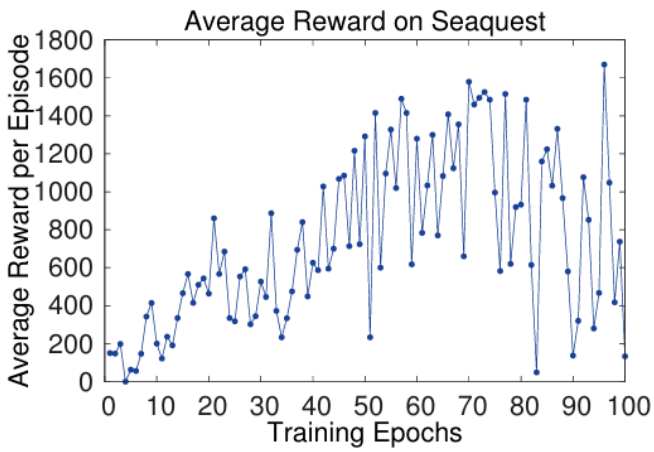
Training and Stability

Evaluation metric

1 **Average total Reward**

- policy의 미세한 변화가 state distribution에 큰 영향 -> 점수 변동폭이 커짐
- 매우 noisy가 심함

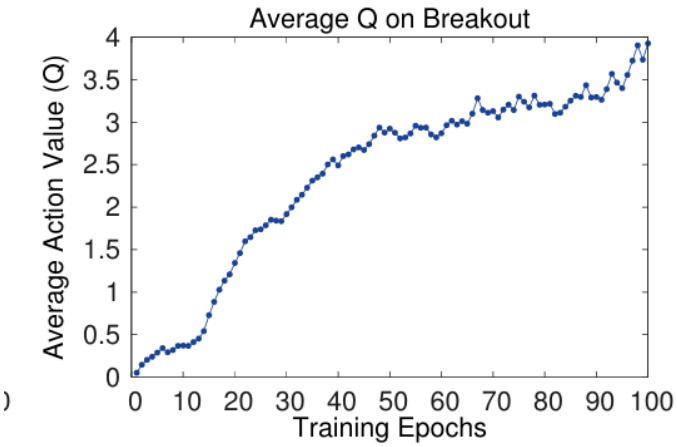


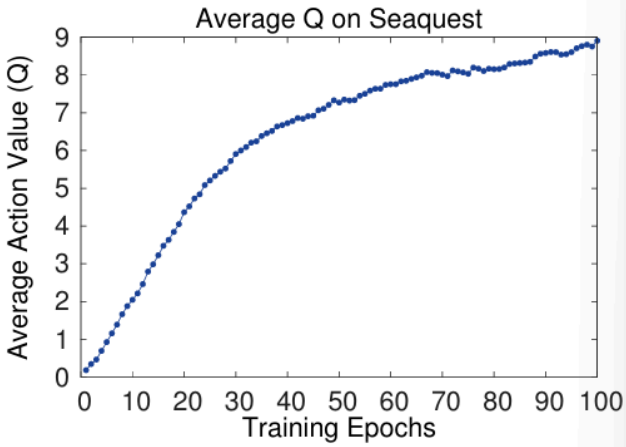


2 **Average predicted** Q

: 에이전트가 받을 것으로 예상하는 Q값누적$rewardexpectation$의 평균

- <u>발산하지 않고 안정적임</u>





Evaluation

	B. Rider	Breakout	Enduro	Pong	Q*bert	Seaquest	S. Invaders
Random	354	1.2	0	−20.4	157	110	179
Sarsa [3]	996	5.2	129	−19	614	665	271
Contingency [4]	1743	6	159	−17	960	723	268
DQN	**4092**	**168**	**470**	**20**	**1952**	**1705**	**581**
Human	7456	31	368	−3	18900	28010	3690
HNeat Best [8]	3616	52	106	19	1800	920	**1720**
HNeat Pixel [8]	1332	4	91	−16	1325	800	1145
DQN Best	**5184**	**225**	**661**	**21**	**4500**	**1740**	1075

Table 1: The upper table compares average total reward for various learning methods by running an ϵ-greedy policy with $\epsilon = 0.05$ for a fixed number of steps. The lower table reports results of the single best performing episode for HNeat and DQN. HNeat produces deterministic policies that always get the same score while DQN used an ϵ-greedy policy with $\epsilon = 0.05$.

공감

letterhill 님의 블로그

letterhill 님의 블로그 입니다.

댓글 0

letterhill

내용을 입력하세요.

등록

DESIGN BY TISTORY | 관리자